SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Concept Mapping

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO1: *Apply cloud computing technologies including hardware-software evolution, virtualization, web services, and service models to develop cloud-based solutions.* (BL2)

Dave's Taxonomy: Imitation (Level 1) **Total Marks: 50**
Question Count: 4

Code of Conduct

I P.Vishwa Nandan (192473015) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: vishwanandan

Assessment Tasks

Concept Mapping Tasks

Create concept maps for the following tasks. Each map must show key nodes, link labels, and a logical hierarchy.

1. Concept Map 1: Cloud Computing Fundamentals. Include definition, characteristics, benefits, and limitations.
2. Concept Map 2: Evolution of Cloud Computing. Connect hardware evolution, internet software evolution, distributed systems, and service delivery.
3. Concept Map 3: Service Models. Map IaaS, PaaS, SaaS, and XaaS with provider responsibilities and user responsibilities.
4. Concept Map 4: Virtualization and Web Services as Enablers of Cloud. Show how these two concepts support cloud deployment and scalability.

Expected concept nodes:

- Elasticity, scalability, resource pooling, on-demand access, measured service
- Virtual machine, hypervisor, abstraction, API, SOAP, REST, provider, consumer
- Infrastructure layer, platform layer, application layer, shared responsibility

Concept Mapping Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Identification	25%	All major concepts are accurately identified	Most concepts are identified correctly	Limited concepts are identified	Essential concepts are missing
Relationship Mapping	25%	Links between concepts are accurate and meaningful	Most links are correct	Some links are weak or unclear	Relationships are mostly incorrect
Hierarchy and Organization	20%	Excellent structure with clear hierarchy	Mostly organized with minor issues	Basic structure with weak hierarchy	No logical organization
Technical Relevance	20%	Map strongly reflects cloud course content	Mostly aligned to topic	Partially aligned to topic	Weak alignment to topic
Visual Clarity	10%	Neat, readable, and easy to interpret	Generally readable	Somewhat cluttered	Poorly presented

Total Marks:

Permitted Tools :

draw.io / diagrams.net, Lucidchart, MindMeister, Miro, Canva, PowerPoint, Google Drawings, or equivalent diagramming tool

Procedure for Preparing the Digital Concept Map

1. Use the following procedure to prepare the diagram and upload evidence.
2. Students who already know another diagramming tool may use it, but the exported evidence and source file must still be submitted.
3. Open https://app.diagrams.net/ or the desktop version of draw.io / diagrams.net.
4. Choose Device, Google Drive, or OneDrive as the save location according to availability.
5. Create a blank diagram and name the file as CSA15_CO1_AT1_RegisterNumber_Concept_Map.drawio.
6. Place the concept map title at the center or as a clearly visible anchor node.
7. Add major cloud nodes: cloud definition, cloud evolution, cloud architecture, service models, deployment models, virtualization, web services, SOAP, REST, APIs, elasticity, scalability, resource pooling, measured service, and emerging paradigms.
8. Connect nodes with labeled arrows such as uses, depends on, enables, scales through, communicates through, belongs to, supports, provides, consumes, or controls.
9. Use colors meaningfully: one color for service models, one for cloud characteristics, one for API/web-service concepts, and one for concept map title-specific elements.
10. Export the final diagram as PNG or PDF. Keep the editable .drawio source file also.
11. Paste the exported diagram image or screenshot in this DOCX under Task 1.
12. Upload the DOCX, exported PNG/PDF, and .drawio file to Google Classroom.

Task 1: Concept Map

Create a concept map in draw.io, Lucidchart, Miro, Figma, or any suitable diagramming tool. The map must connect cloud ecosystem, cloud actors, service models, deployment models, virtualization, web services, REST/SOAP APIs, scalability, elasticity, and the student's Project.

Task 2A: Service Model Responsibility Matrix

Prepare a technical matrix that compares SaaS, PaaS, IaaS, FaaS/serverless, database-as-a-service, storage-as-a-service, identity-as-a-service, and monitoring/logging services. For each selected service category, explain what is managed by the cloud provider, what is managed by the student application team, and why that split fits the Project.

Task 2B: Cloud Service Decision Matrix

Map Project modules to cloud services. Include authentication, frontend hosting, backend/API layer, database, object storage, analytics/logging, notification, AI/Python module, security, CI/CD, backup, and monitoring. Students should justify the most relevant service choices instead of forcing every possible cloud service into the design.

Task 3: API Flow, Elasticity, and Scalability Design

Draw a professional cloud-flow diagram for the Project modules. The diagram must show user app or web app, API gateway/backend service, authentication, compute layer, database, object storage,

analytics/logging, monitoring, security boundaries, and scaling points. Students may replicate the given visual pattern, but all components and labels must be customized to their own architecture.

Task 4: Technical Justification

Write a concise justification explaining how the diagram and matrices prove CO1. The justification must discuss service model selection, API role, elasticity, scalability, virtualization or managed compute choice, and cloud responsibility boundaries.

1. Evaluate the effectiveness of your concept map in representing cloud computing principles. Which design decisions most strongly support your solution, and why?
2. Analyze how scalability, elasticity, and resource management influence the architecture shown in your concept map. What challenges could arise if these capabilities were absent?
3. Justify the selection of the APIs, web services, or cloud services included in your concept map. How do they contribute to the overall functionality and efficiency of the system?
4. Critically examine the relationships between the components in your concept map. How do these interactions improve reliability, performance, or user experience?
5. Reflect on the process of developing your concept map. How has creating and analyzing the map changed your understanding of cloud computing architecture, and what improvements would you make if you were redesigning it?

Expected Digital Evidence

- Editable diagram source file and exported PNG/PDF for the concept map.
- Completed service model responsibility matrix and cloud service decision matrix.
- Architecture/API flow diagram exported as PNG/PDF.
- README file containing title, tools used, repository structure, and a short CO1 reflection.

GitHub and Google Classroom Submission

1. Create a repository using a clear name: CSA15_CO1_AT<Number>_<RegisterNumber>.
2. Upload editable source files, exported PDF/PNG evidence, screenshots, and a short README explaining the work.
3. Keep filenames professional and include the register number where appropriate.
4. Submit only the GitHub repository link in Google Classroom before the deadline.
5. Ensure the repository is accessible to the faculty evaluator during assessment.

Implementation Note

Faculty may evaluate the submitted evidence digitally using repository inspection, diagram quality, oral explanation, and CO-aligned rubric scoring. The focus is on technical understanding, cloud design reasoning, and hands-on evidence rather than routine written work.

Task 2A: Service Model Responsibility Matrix

Project Title: Java-Based Smart Campus Resource Allocation and Reservation System

Objective: To compare different cloud service categories based on cloud provider responsibilities, student application team responsibilities, and their suitability for the Smart Campus project.

S. No.	Service Category	Cloud Provider Responsibilities	Student Application Team Responsibilities	Why This Split Fits the Project
1	**SaaS (Software as a Service)**	Manages the complete software application, servers, operating system, maintenance, and updates.	Configures the service, manages users and permissions, and uses the application to support project activities.	Ready-to-use email or collaboration software can support communication between students, faculty, and administrators.
2	**PaaS (Platform as a Service)**	Manages servers, networking, operating systems, runtime environments, and platform maintenance.	Develops, tests, deploys, and maintains the Java Spring Boot application and manages application data.	Simplifies backend deployment and allows the team to focus on reservation logic and resource management instead of server maintenance.
3	**IaaS (Infrastructure as a Service)**	Manages physical servers, data center facilities, networking hardware, and virtualization infrastructure.	Manages the operating system, application runtime, software installation, application security, and deployed application.	Provides greater control over the environment when deploying and configuring the Java backend.
4	**FaaS / Serverless (Function as a Service)**	Manages the server infrastructure, function execution environment, and automatic scaling of functions.	Writes and deploys functions, configures triggers and permissions, and manages function	Supports event-driven tasks such as sending booking confirmations, processing notifications, and

			logic and dependencies.	responding to reservation events.
5	**Database as a Service (DBaaS)**	Manages database infrastructure, database engine maintenance, software patching, and configured backup and recovery features.	Designs database tables, writes SQL queries, manages reservation records, and configures data access permissions.	Stores student details, faculty details, resource information, booking records, and resource availability.
6	**Storage as a Service**	Provides and maintains storage infrastructure and the selected durability, availability, and replication features.	Organizes files, manages uploads, configures access permissions, and defines file retention and lifecycle requirements.	Provides storage for uploaded documents, resource images, and other files associated with reservations.
7	**Identity as a Service (IDaaS)**	Provides and maintains identity management and authentication services.	Configures authentication, user roles, access policies, and application-level authorization for students, faculty, and administrators.	Supports secure login and role-based access control so users can access only the functions permitted by their roles.
8	**Monitoring and Logging Services**	Provides tools and infrastructure for collecting, storing, analyzing, and displaying logs and monitoring metrics.	Adds application logging, configures dashboards and alerts, monitors application errors, and investigates performance issues.	Helps detect failed reservations, backend errors, slow responses, and other problems affecting system reliability.

Conclusion

The responsibility matrix identifies the responsibilities shared between the cloud provider and the student application team. The provider manages the underlying infrastructure and managed services according to the selected service model, while the team develops the application, manages project data, and configures security and application-specific settings. Selecting suitable cloud services can reduce infrastructure maintenance, improve scalability, and support reliable operation of the Smart Campus Resource Allocation and Reservation System.

Task 2B: Cloud Service Decision Matrix

Project Title: Java-Based Smart Campus Resource Allocation and Reservation System

Objective: To map the project modules to appropriate cloud services and justify their selection based on functionality, security, scalability, and cost-effectiveness.

Cloud Service Decision Matrix

S. No.	Project Module	Selected Cloud Service	Purpose	Justification
1	Authentication	Amazon Cognito	Manages user sign-up, sign-in, and authentication.	Supports secure authentication for students, faculty, and administrators.
2	Frontend Hosting	Amazon S3 and Amazon CloudFront	Stores and delivers the HTML, CSS, and JavaScript frontend.	Enables users to access the web application through the internet with efficient content delivery.
3	Backend/API Layer	AWS Elastic Beanstalk	Hosts the Java Spring Boot backend and REST APIs.	Simplifies application deployment and reduces the need to manage the underlying infrastructure directly.
4	Database	Amazon RDS for MySQL	Stores user details, resource information, booking records, and availability.	Provides a managed relational database suitable for maintaining structured campus reservation data.
5	Object Storage	Amazon S3	Stores uploaded documents, resource images, and other files.	Provides scalable storage without requiring the application team to maintain a separate file server.
6	Analytics and Logging	Amazon CloudWatch Logs and MySQL queries	Collects application logs and supports basic analysis of reservation data.	Helps identify application errors and generate reports about resource usage and booking activity without introducing a separate analytics platform.

7	Notification	Amazon Simple Email Service (SES)	Sends booking confirmations, approval messages, and reminders through email.	Keeps students and faculty informed about reservation status and approval decisions.
8	AI/Python Module	Not required in the initial version; AWS Lambda can be considered for future Python-based processing.	Could support future predictive analysis of resource demand.	The current system focuses on resource allocation and reservations, so an AI module is unnecessary unless predictive or intelligent allocation features are added.
9	Security	AWS Identity and Access Management (IAM), security groups, and HTTPS	Controls cloud resource permissions, restricts network access, and protects data transmission.	Helps protect user information, reservation records, and application resources from unauthorized access.
10	CI/CD	AWS CodePipeline and AWS CodeBuild	Automates application build, testing, and deployment workflows.	Reduces repetitive manual deployment work and helps deliver tested application updates consistently.
11	Backup	Amazon RDS automated backups and Amazon S3 Versioning	Supports database recovery and helps preserve previous versions of stored files.	Helps recover important reservation records and files after accidental deletion or other data-loss incidents.
12	Monitoring	Amazon CloudWatch Metrics, Dashboards, and Alarms	Tracks application performance, resource utilization, and service health.	Helps identify failures and performance issues before they significantly affect users.

Task 4: Technical Justification

1. Effectiveness of the Concept Map & CO1 Alignment The concept maps and architectural diagrams directly satisfy CO1 by distinctly visualizing cloud service models (IaaS, PaaS, SaaS) and their corresponding shared responsibility boundaries. The decision to visually separate underlying virtualization (hardware abstraction via hypervisors) from web services (REST API integration) is highly effective. It proves how theoretical cloud principles translate into structural deployment choices, specifically showing where a provider's physical infrastructure management ends and the consumer's application control begins.

2. Impact of Scalability, Elasticity, and Resource Management Resource pooling serves as the foundation for the architecture's dynamic workload management. Elasticity is implemented via the Auto-Scaling Group, which monitors CPU metrics to instantly scale backend compute instances during unexpected traffic spikes (e.g., sudden surges in rental bookings). Scalability is achieved at the data layer through asynchronous read replicas, accommodating long-term growth in database queries. Absent these capabilities, the system would suffer severe latency during peak usage and risk complete database locking during concurrent catalog browsing and transaction processing.

3. Justification of APIs and Managed Cloud Services REST APIs function as the critical decoupling layer between the public-facing application and the secure backend compute instances. Routing all traffic through an API Gateway enables centralized authentication (IAM) and traffic filtering without exposing virtual machines to the public internet. Furthermore, selecting managed cloud services—such as Object Storage for KYC documents and vehicle images, alongside a centralized logging service for analytics (e.g., Tableau integration)—offloads non-transactional processing from the core compute layer, dramatically improving overall system efficiency.

4. Component Relationships and System Optimization The interaction between the Edge Security Boundary and the Private VPC maximizes both reliability and user experience. By positioning a Web Application Firewall (WAF) ahead of the API Gateway, malicious traffic is dropped before consuming backend compute resources. Internally, decoupling the primary transactional database (handling write-heavy booking logic) from the read replicas (handling read-heavy user catalog searches) ensures high-speed querying, preventing performance degradation and ensuring a seamless user experience even under heavy load.

5. Reflection and Future Architectural Improvements Developing these diagrams reinforced the practical complexities of translating abstract cloud concepts into functional architectures, particularly regarding security boundaries and managed compute choices. If redesigning this architecture for a more advanced deployment, the primary improvement would be shifting from monolithic compute instances to a microservices architecture. By containerizing individual functions (e.g., isolating user authentication, billing, and fleet management into separate Kubernetes pods), the system could achieve granular elasticity, scaling only the specific services under load rather than duplicating the entire backend environment.